



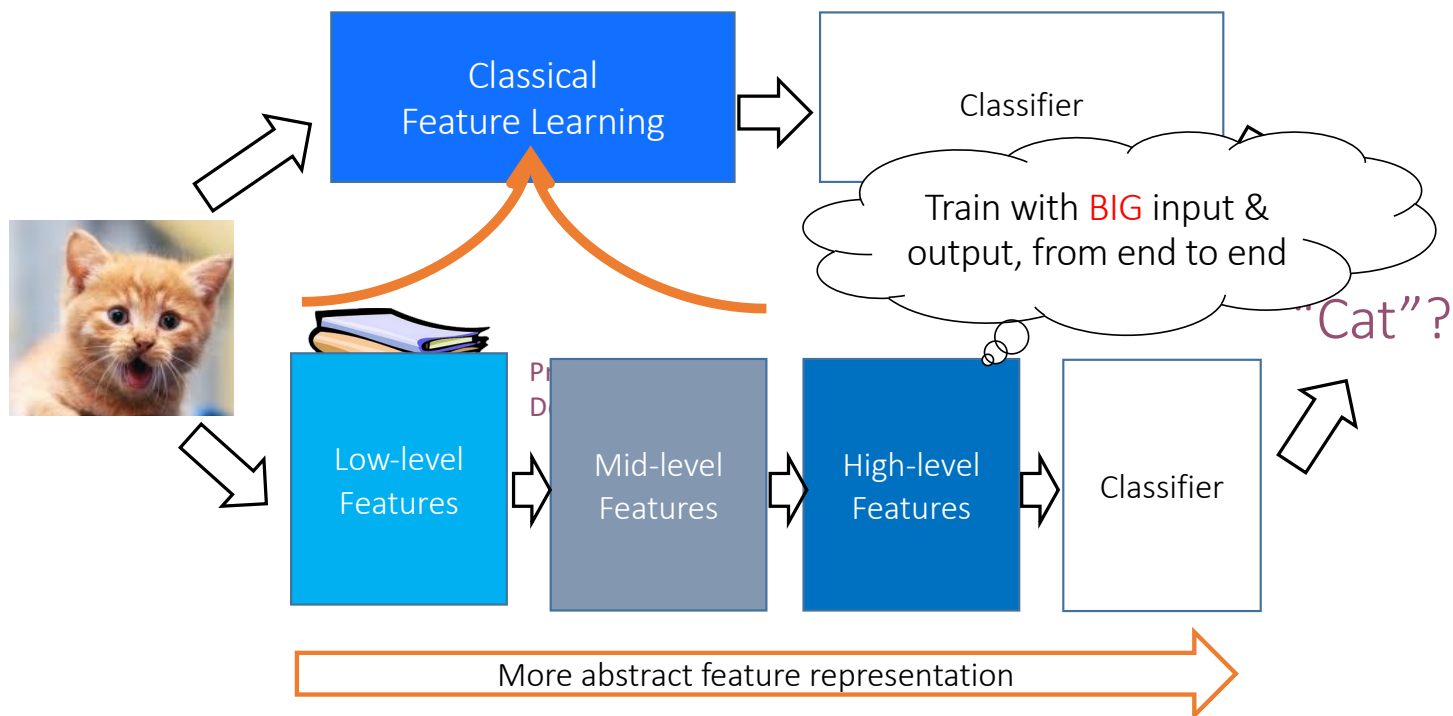
2019 Turing Award winners, left to right, Yann LeCun, Geoff Hinton, and Yoshua Bengio,
reoriented artificial intelligence around neural networks

A Triumph of Deep Learning: 2012 - present

Top-performers in many tasks, over many domains

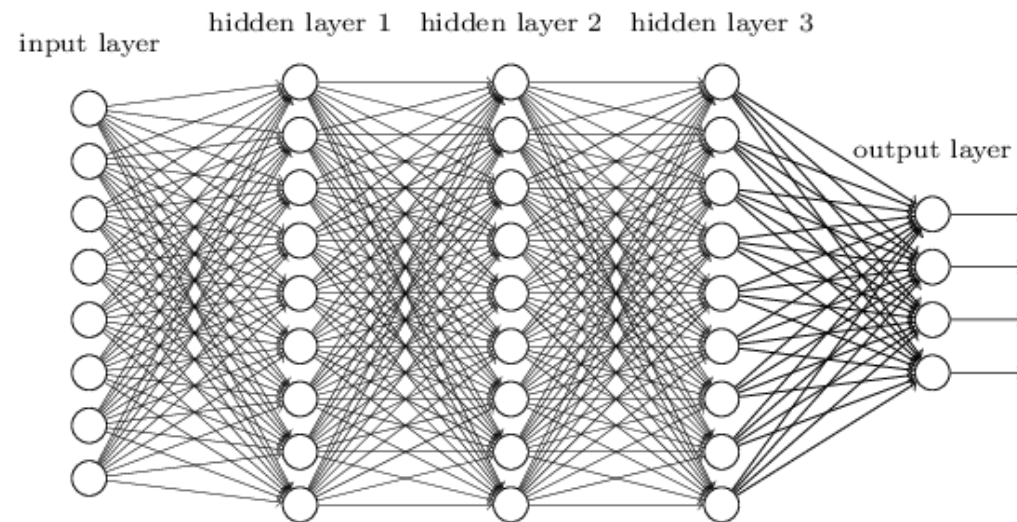


Feature learning: Going Deep



Deep learning

- Learn a *feature hierarchy* all the way from raw inputs (e.g. pixels) to classifier
- Each layer extracts features from the output of previous layer
- Train all layers jointly



Status Quo

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)



ResNet, 152 layers
(ILSVRC 2015)

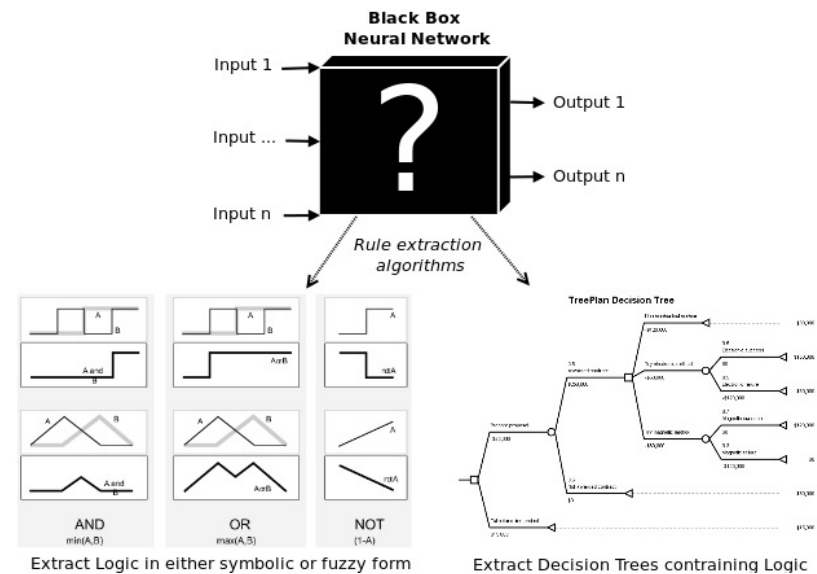


Current Trend:

- To build increasingly larger, deeper networks, trained with more massive data, based on the benefits of high-performance computing.
- Play with the connectivity and add “skips”

Grand Challenges

- Why/how deep learning works?
 - *In theory, many cases shouldn't even work...*
 - Gap between engineering (or art) and science: Lack of theoretical understandings & guarantees, and analytical tools
 - Training is computationally expensive and difficult, relying on many “magics”
 - No principled way to incorporate domain expertise, or to interpret the model behaviors



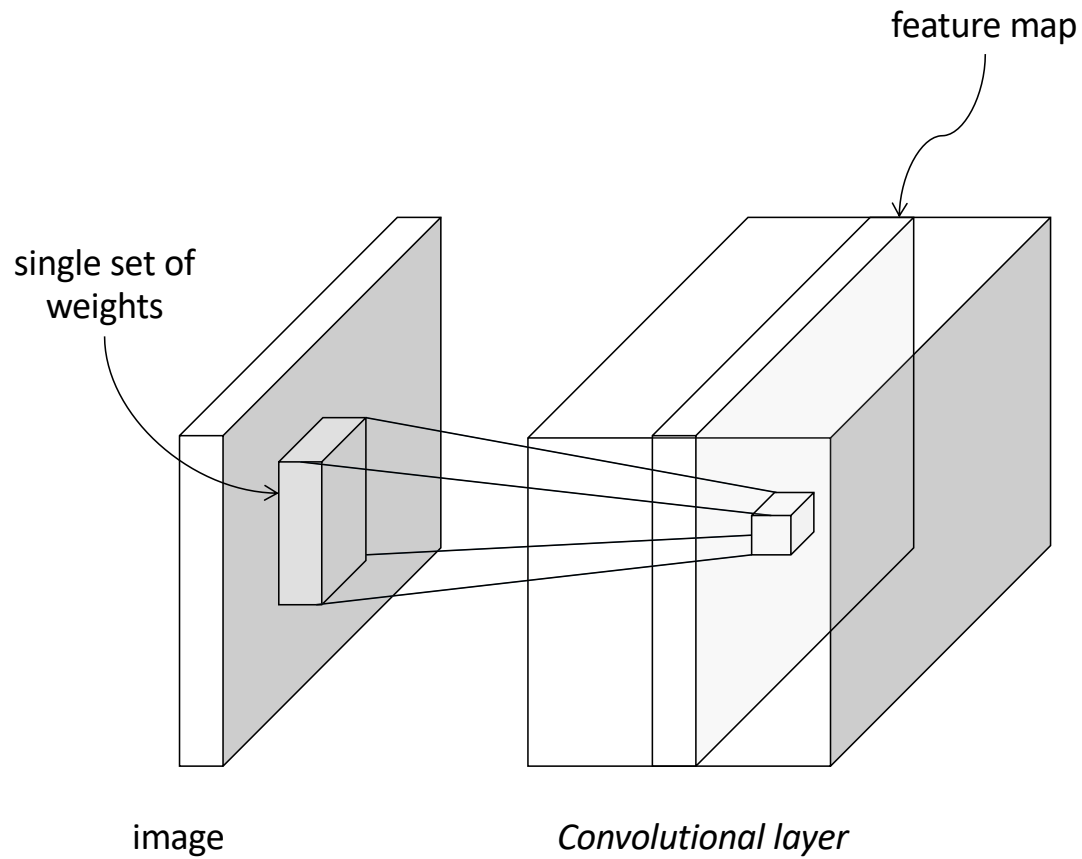
From NNs to Convolution NNs

The most important building block in modern deep learning

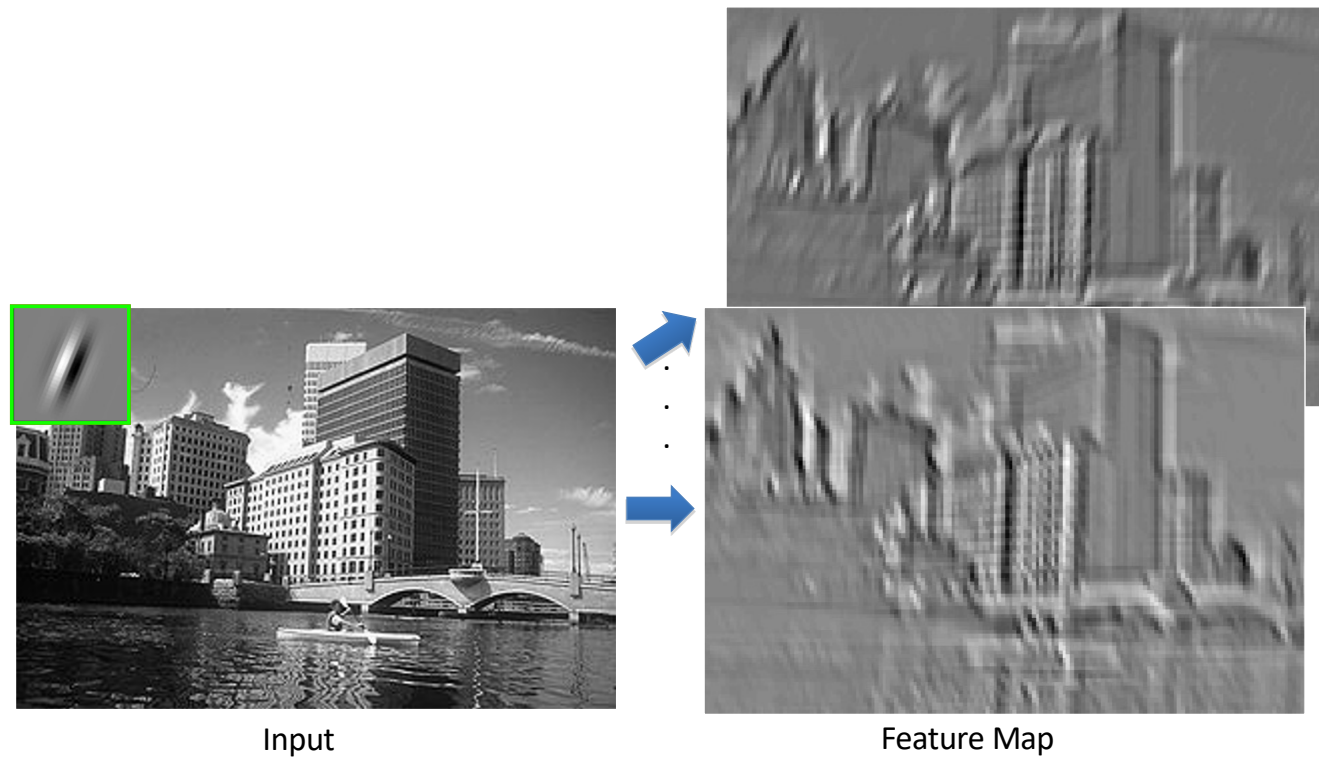
Review: Spatial Low-Level Vision

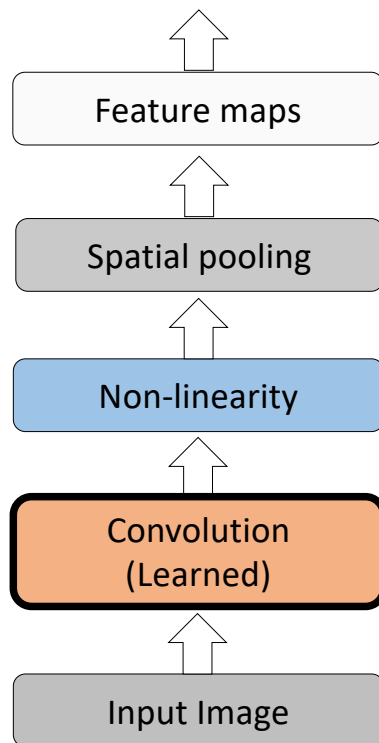
- **Critical Building Block:** Filters and filter banks
 - Often implemented via **convolution (yes, that guy in deep learning)**
 - Detection of edges, corners, and other local features
 - Texture pattern recognition and synthesis
 - Can include multiple orientations
 - Can include **multiple scales:** “filter pyramids” – a key feature in deep learning

From fully connected to convolutional networks



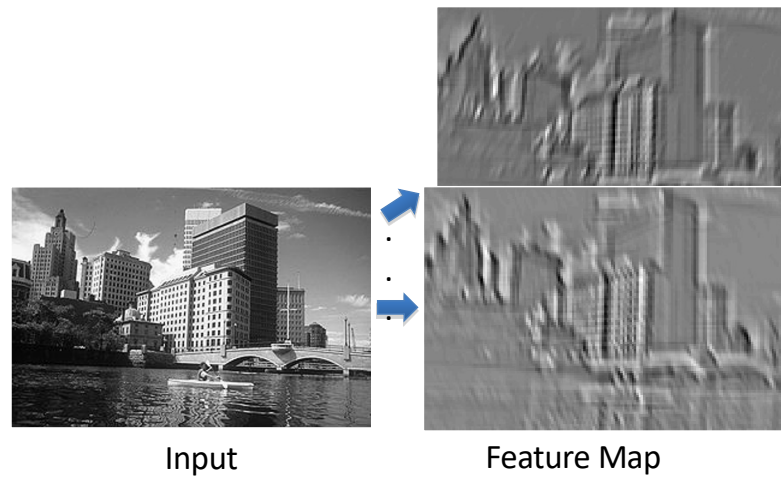
Convolution as feature extraction





Source: R. Fergus, Y. LeCun

Key operations in a CNN



Review: Computer Vision Has “Three Levels”



“There’s an edge!”

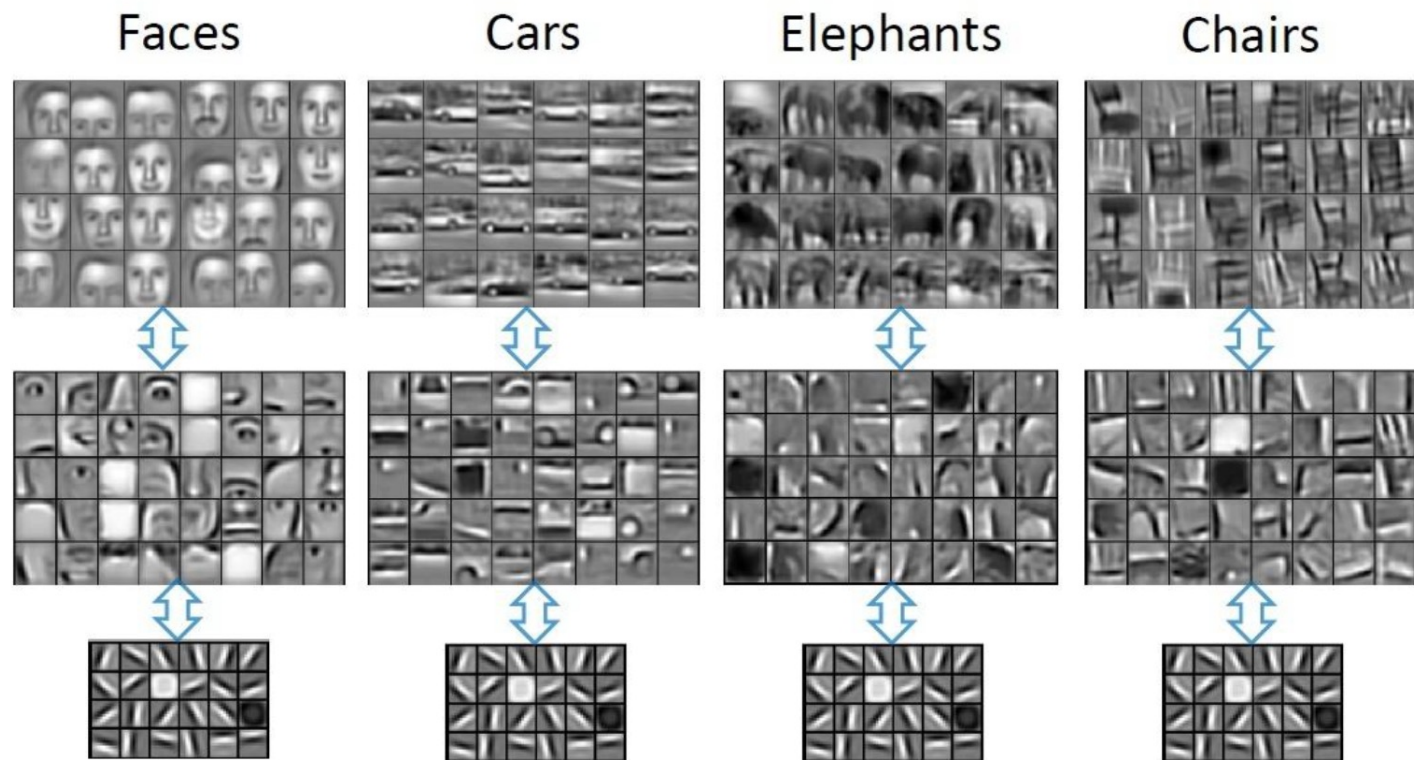


“There’s an object and a background!”

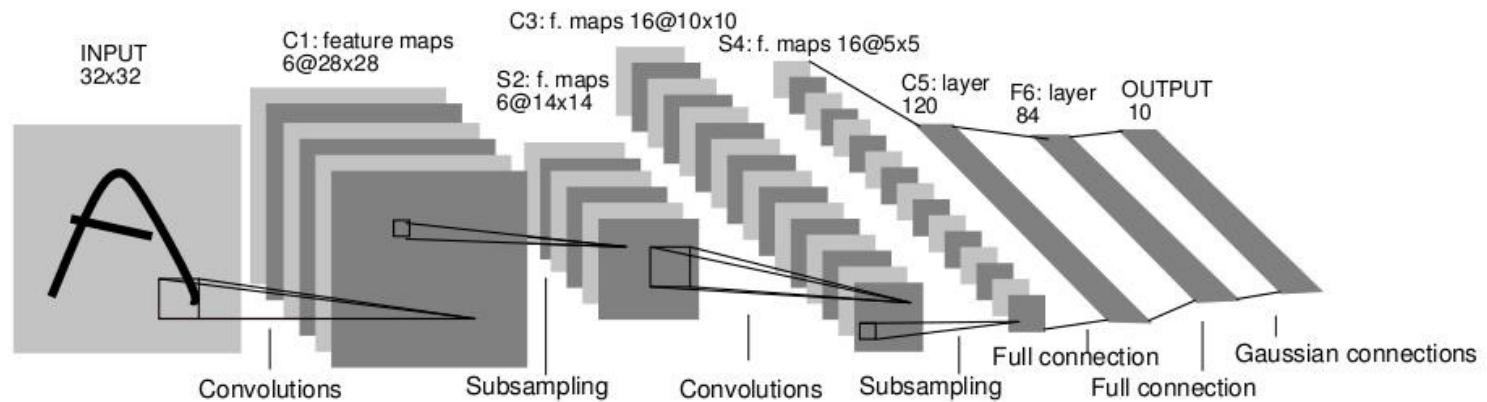


“There’s a chair!”

Deep Features (May) Learn Semantic Hierarchy



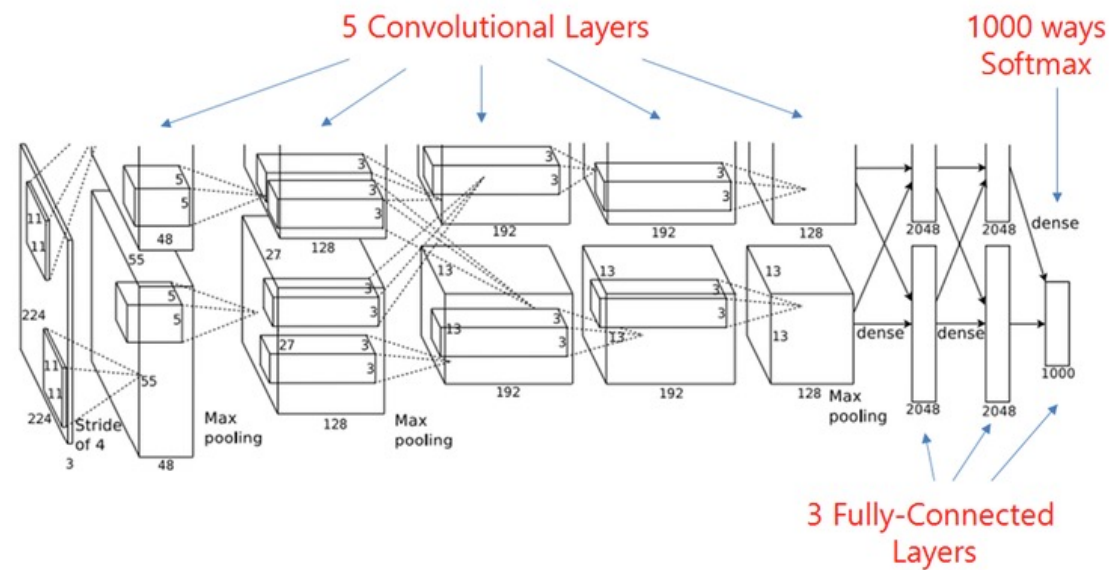
LeNet-5



- Average pooling
- Sigmoid or tanh nonlinearity
- Fully connected layers at the end
- Trained on MNIST digit dataset with 60K training examples

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, [Gradient-based learning applied to document recognition](#), Proc. IEEE 86(11): 2278–2324, 1998.

AlexNet, 2012



- The **FIRST** winner deep model in computer vision, and one of the most classical choices for domain experts to adapt for their applications
- 5 convolutional layers + 3 fully-connected layers + softmax classifier
- Three Key Design Features: ReLU, dropout, data augmentation

Data Augmentation

Horizontal Flip



Crop



Rotate



- Adding noise to the input: a special kind of augmentation
- Be careful about the transformation applied -> **label preserving**
 - **Example:** classifying 'b' and 'd'; '6' and '9'

VGG-Net, 2014

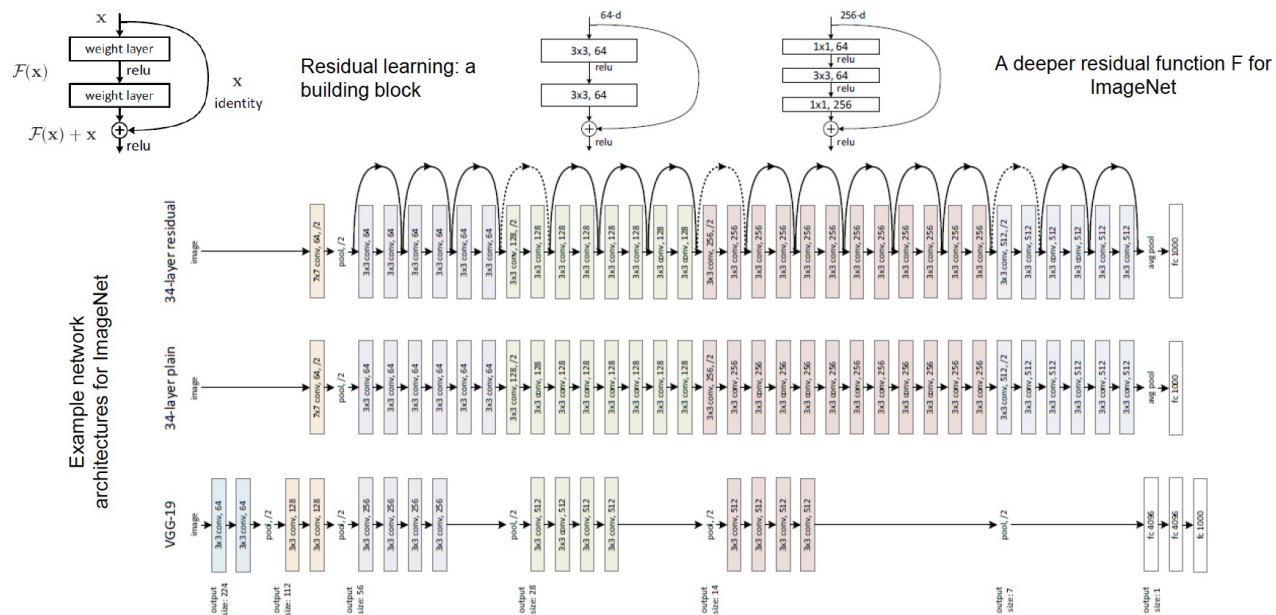
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Key Technical Features:

- Increase depth (up to 19)
- Smaller filter size (3)

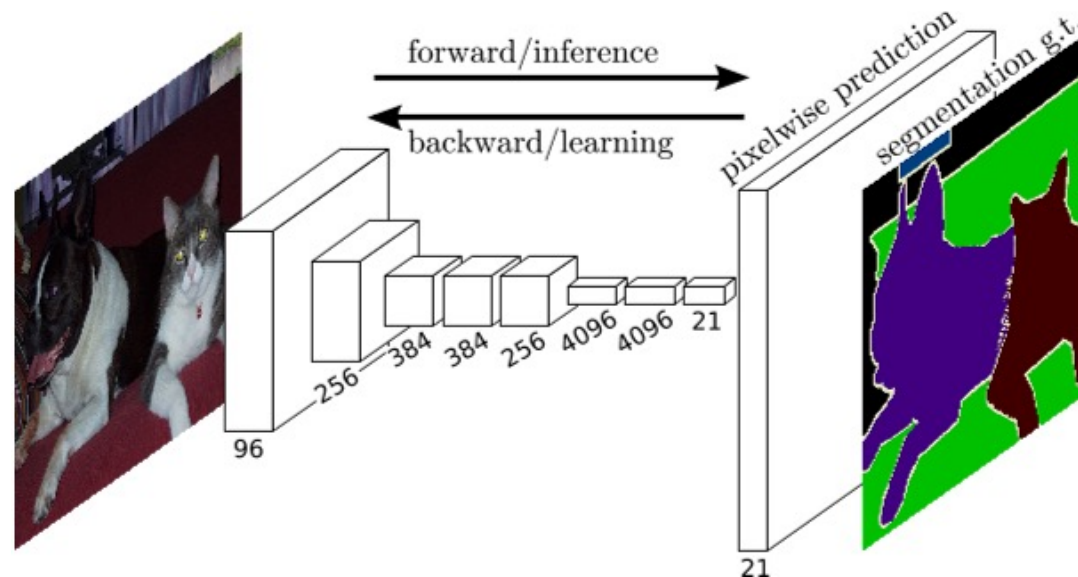
Configurations D and E are widely used for various tasks, called *VGG-16* and *VGG-19*

Deep Residual Network (ResNet), 2015



Key Technical Features: skip connections for residual mapping, up to > 1000 layers

Fully Convolutional Network (FCN), 2014

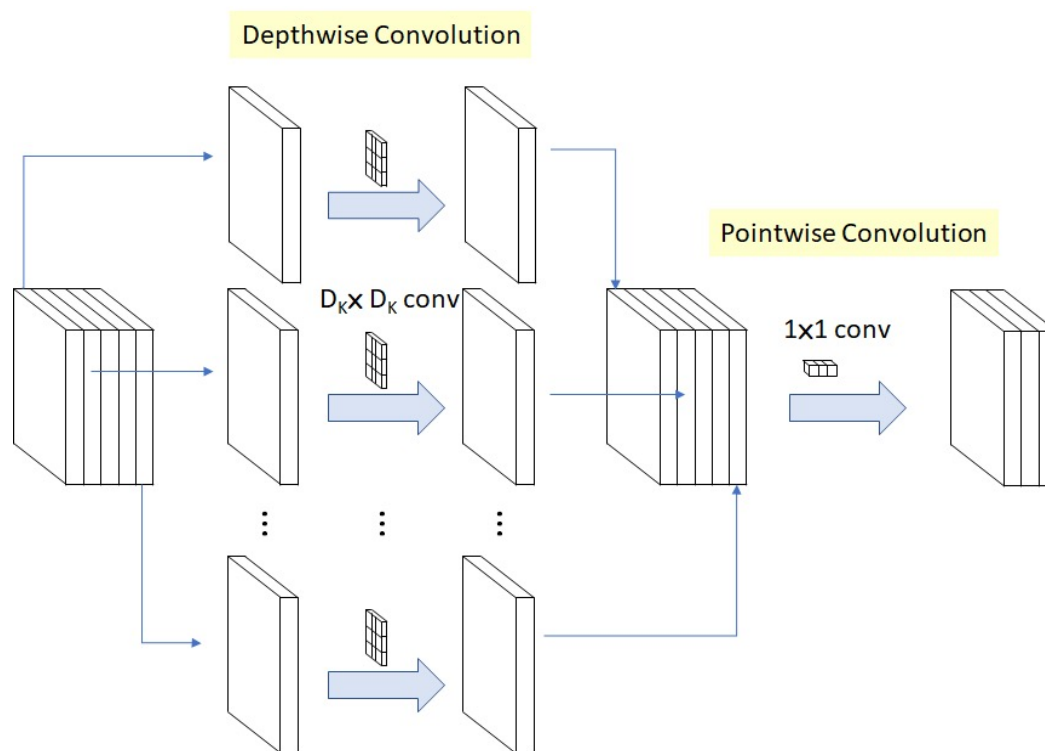


Key Technical Features:

- No fully-connected layer -> No fixed requirement on input size
- Widely adopted in pixel-to-pixel prediction tasks, e.g., image segmentation

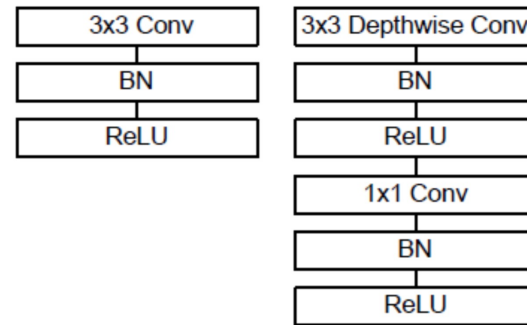
Depth-Wise Convolution

- **Depthwise convolution** is the channel-wise spatial convolution.
- It is often used together with **pointwise convolution**, i.e., 1×1 convolution to change the channel dimension (number of feature maps)



MobileNet (v1)

- Single streamlined, very light-weight architecture
- **Main idea:** Depthwise Separable Convolutions
- **Other ideas:** Width Multiplier α for Thinner Models + Resolution Multiplier ρ for Reduced Representation

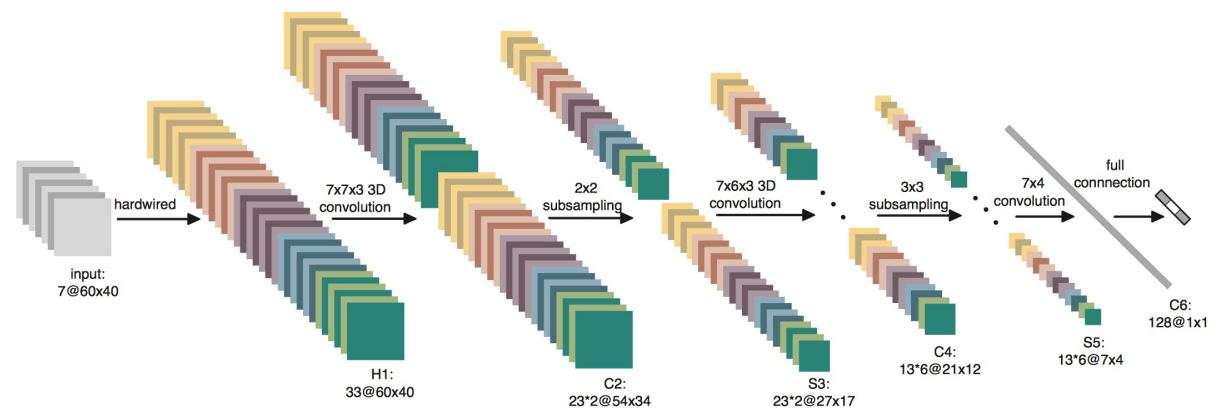


Standard Convolution (Left), Depthwise separable convolution (Right) With BN and ReLU

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size	
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$	
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$	
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$	
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$	
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$	
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$	
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$	
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$	
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$	
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$	
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$	
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$	
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$	
5x	Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
	Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
	Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
	Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
	FC / s1	1024×1000	$1 \times 1 \times 1024$
	Softmax / s1	Classifier	$1 \times 1 \times 1000$

3D Convolutional Network (3D CNN), 2011



Key Technical Features:

- Going from 2D convolutional filters to 3D filters, to take temporal coherence into consideration

Basics: Transformer in NLP

- Standard model in NLP tasks
- Only consists of self-attention modules, instead of RNN
- Encoder-decoder
- Requires large dataset and high computational cost
- Pre-training and fine-tuning approaches : BERT & GPT

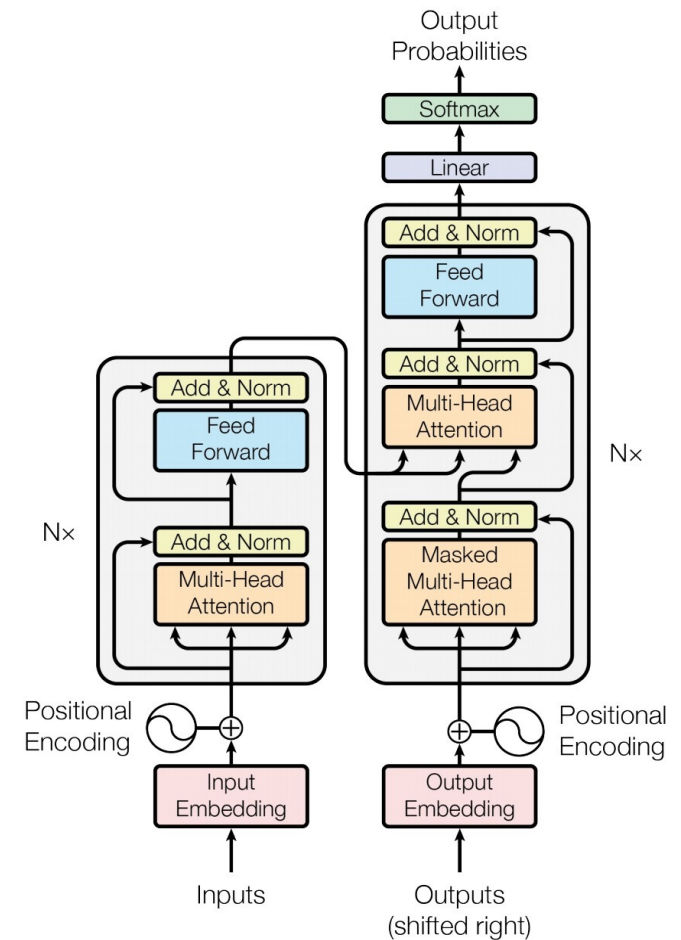
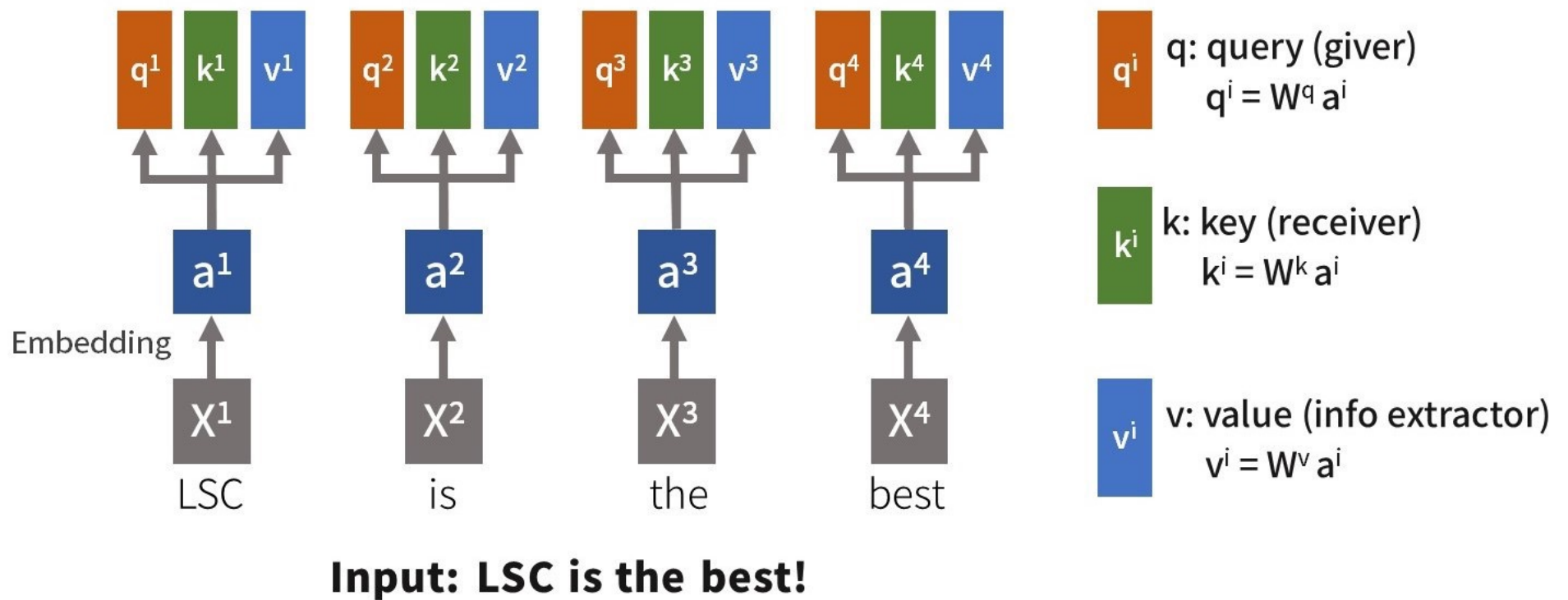
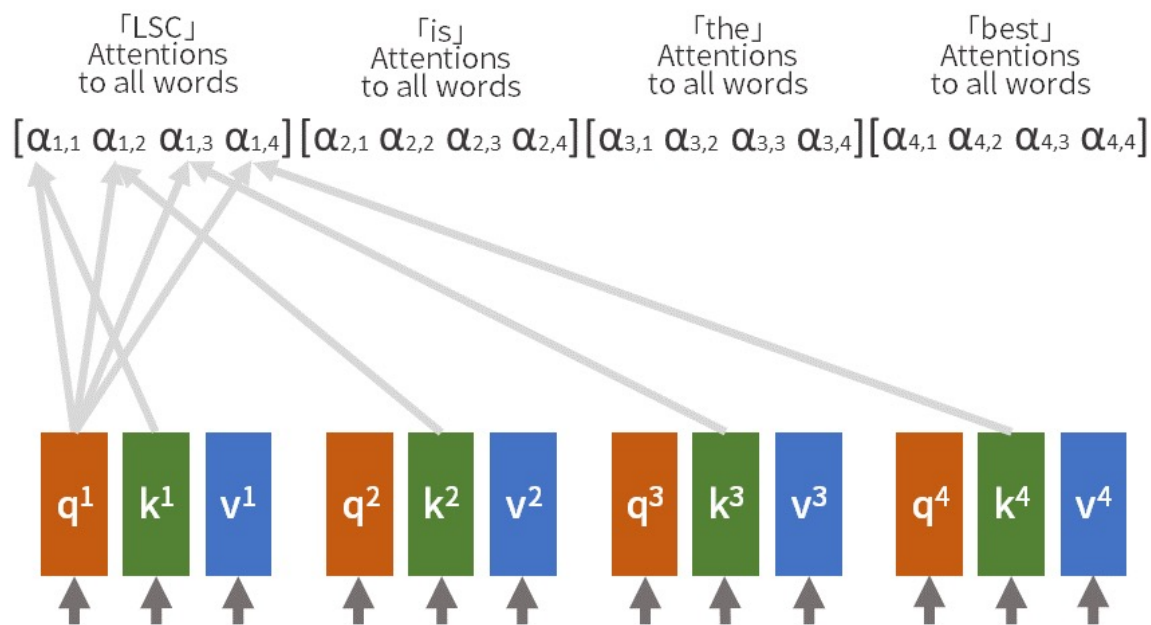


Figure 1: The Transformer - model architecture.

Basics: Self-Attention



Basics: Self-Attention



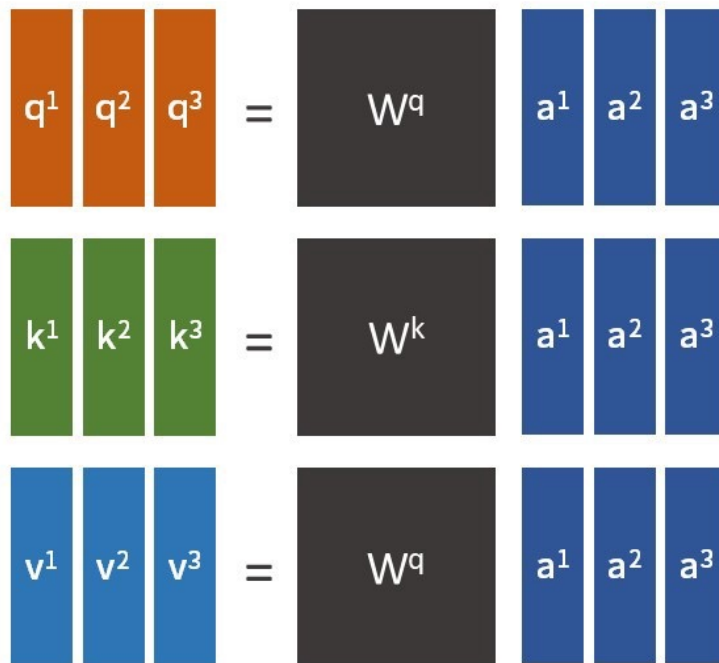
$$\alpha_{i,j} = \frac{q^i \cdot k^j}{\sqrt{d}}$$

d: dimension of q, k

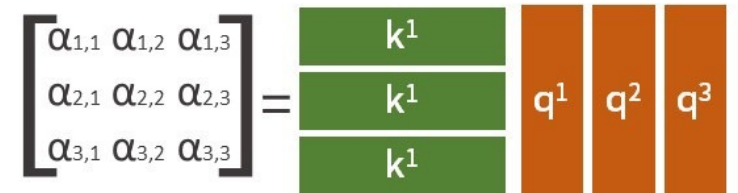
$$A = \begin{bmatrix} \alpha_{1,1} & \alpha_{1,2} & \alpha_{1,3} & \alpha_{1,4} \\ \alpha_{2,1} & \alpha_{2,2} & \alpha_{2,3} & \alpha_{2,4} \\ \alpha_{3,1} & \alpha_{3,2} & \alpha_{3,3} & \alpha_{3,4} \\ \alpha_{4,1} & \alpha_{4,2} & \alpha_{4,3} & \alpha_{4,4} \end{bmatrix}$$

Attention Matrix

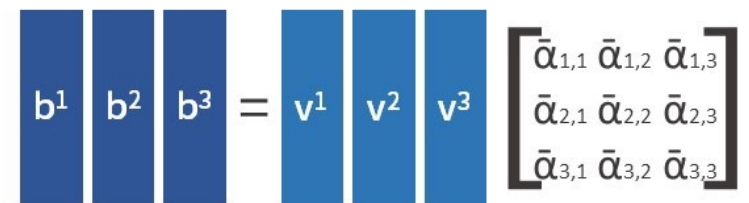
Basics: Self-Attention



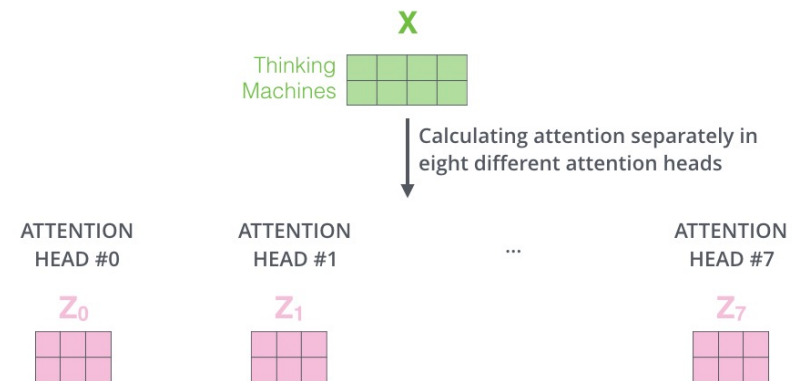
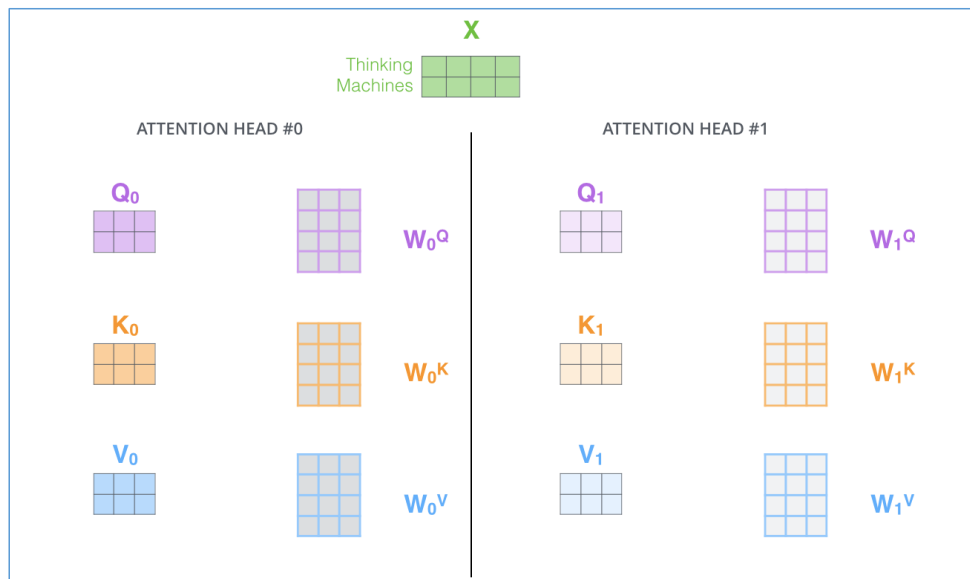
Attention A:



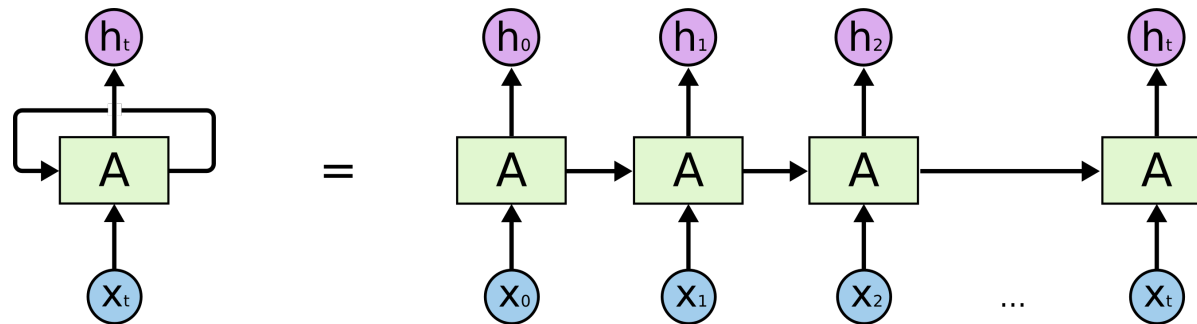
Output:



Multi-Headed Attention



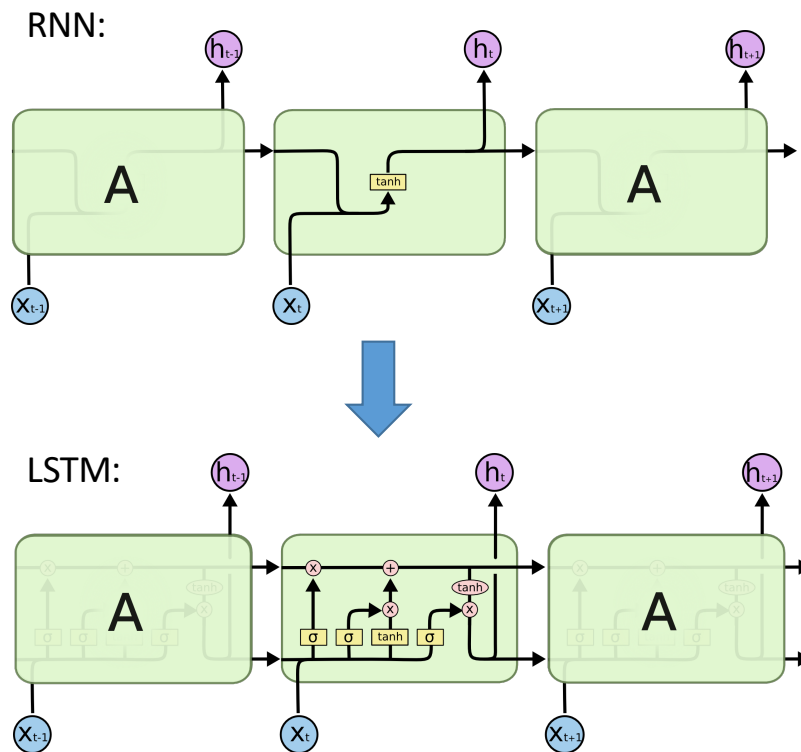
RNN and LSTM



- A RNN is **unfolded** its forward and backward computations.
- Backpropagation Through Time (**BPTT**): Because the parameters are shared by all time steps in the network, the gradient at each output depends not only on the calculations of the current time step, but also the previous time steps
- **Vanishing/Exploding Gradients**: Difficulty in learning long-term dependency

An intro article for RNN/LSTM: [“Understanding LSTM Networks”](http://colah.github.io/posts/2015-08-Understanding-LSTMs/):
<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

RNN and LSTM



- A Long Short Term Memory (LSTM) combats vanishing gradients through a **gating** mechanism, thus capturing **long-term dependency** better.
 - Similar “shortcut” idea to ResNet
- An LSTM does the exact same thing as a RNN, just in a different way!
- **Key Idea:** the gating functions are learned together with weights, and determine how much information we would like keep from last state and current computation, etc.